



MINI PROJECT: BANKING MANAGEMENT

CS23332-DATABASE MANAGEMENT SYSTEM

Submitted by:

CHARAN SUNDAR V - 241001036

DIVYA S - 241001056

DIVYASHREE S - 241001057

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM
(An Autonomous Institution)



RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project titled "Bank Management" is the Bonafide work of "CHARAN SUNDAR V (241001036), DIVYA S (241001056), DIVYASHREE S(241001057)" who carried out the project work under my supervision.

SIGNATURE

Dr.Valarmathie

HEAD OF THE DEPARTMENT

Information Technology
Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam
Chennai – 602105

SIGNATURE

Mrs.Ragavardhini

COURSE INCHARGE

Information Technology
Rajalakshmi Engineering College
Rajalakshmi Nagar, Thandalam
Chennai – 602105

This project is submitted for CS23332– DATABASE MANAGEMENT SYSTEM
held on

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

ABSTRACT

This presentation provides an overview of the architecture and core components required to build a simple yet functional Banking Management System using Java, SQL, and JDBC. It explains how Java serves as the application layer for implementing business logic, SQL manages secure data storage and retrieval, and JDBC acts as the bridge enabling seamless communication between the application and the database. Key modules such as account management, transaction processing, user authentication, and data validation are highlighted. The presentation also outlines essential protocols, including database connectivity, error handling, and security measures. Overall, it offers a structured approach for designing a scalable and reliable banking system using widely adopted technologies.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our chairman **Mr.S. Meganathan,B.E., F.I.E** for his sincere endeavour in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan**, for her enthusiastic motivation which inspired us a lot in completing this project and Vice-Chairman Mr. Abhay Shankar Meganathan B.E., M.S., for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college principal, **Dr.S.N.Murugesan M.E., PhD.**, for his kind support and facilities to complete our work on time. We extend heartfelt gratitude to **Dr.P.Valarmathie, Professor and Head of the Department of Information Technology** for her guidance and encouragement throughout the work. We are very glad to thank our course faculty **Mrs.Ragavardhini** of our department for their encouragement and support towards the successful completion of this project. We extend our thanks to our parents, friends, all faculty members, and supporting staff for their direct and indirect involvement in the successful completion of the project for their encouragement and support.

CHARAN SUNDAR V
DIVYA S
DIVYASHREE S

Contents

1	INTRODUCTION	1
1.1	INTRODUCTION.....	1
1.2	SCOPE OF THE WORK.....	1
1.3	PROBLEM STATEMENT.....	1
1.4	AIM AND OBJECTIVES OF THE PROJECT	2
2	SYSTEM SPECIFICATIONS	3
2.1	HARDWARE SPECIFICATIONS.....	3
2.2	SOFTWARE SPECIFICATIONS.....	3
3	MODULE DESCRIPTION	4
3.1	Database Schema and ER Diagram	4
3.1.1	Table:Customer.....	5
3.1.2	Table:Account.....	5
3.1.3	Table:Transaction.....	6
3.2	User Authentication Module.....	6
3.3	Dashboard Module.....	6
3.4	Customer Management Module.....	7
3.5	Account Management Module.....	7
3.6	Transaction Management Module.....	7
3.7	Balance Enquiry Module.....	7
3.8	Static and Base Templates Module.....	7
3.9	Review / Logging Module.....	7
4	CODING	
4.1	Backend Logic.....	8
4.2	Frontend Logic.....	9
5	SCREENSHOT	
6	CONCLUSION AND FUTURE ENHANCEMENTS	
6.1	Future Enhancements.....	15
	REFERENCES	16

Chapter 1

INTRODUCTION

1.1 INTRODUCTION

Banking systems today rely heavily on automation to increase efficiency, accuracy, and security. This project implements a simple yet functional Banking Management System using Java, JDBC, and MySQL. The system allows customers to create accounts, deposit money, withdraw funds, and check balances.

1.2 SCOPE OF THE WORK

The project encompasses essential banking functionalities including adding customers, creating accounts, managing deposits and withdrawals, and viewing balances. Every transaction is meticulously recorded in a database, ensuring data security and consistency via JDBC, and includes basic authentication logic for enhanced reliability.

1.3 PROBLEM STATEMENT

Music listeners often lack a structured, personalized, and visually engaging method to document and track their album ratings and reviews. Existing social media platforms are often cluttered, and local file storage lacks portability and a centralized database. The AlbumLog system solves this by creating a dedicated, user-centric web application for music logging.

1.4 AIM AND OBJECTIVES OF THE PROJECT

To design and implement a simple, secure, and efficient banking management system using Java, JDBC, and MySQL that adhere to the following points:

- Implement CRUD operations for customers and accounts
- Perform deposit, withdrawal, and balance enquiry operations
- Store and track financial transactions
- Use JDBC to securely connect Java and MySQL
- Follow modular programming for easy maintenance
- Provide a scalable base for future advanced banking features

Chapter 2

SYSTEM SPECIFICATIONS

2.1 HARDWARE SPECIFICATIONS

Processor Intel/AMD Dual Core or higher

RAM Minimum 4B

Storage 500MB free space

System Windows / Linux / MacOS

2.2 SOFTWARE SPECIFICATIONS

Java JDK 8+ Programming language

MySQL Server Database storage

JDBC Connector Java-Database communication

Eclipse / IntelliJ IDEA IDE for development

XAMPP / MySQL Workbench Database administration

Chapter 3

MODULE DESCRIPTION

The Banking Management System is structured into multiple modules, each responsible for performing a particular operation. These modules ensure modularity, maintainability, and clarity in the project's workflow

3.1 Database Schema and ER Diagram

The relationships between the core entities are fundamental to the system's design. The ER diagram below illustrates how customers, accounts, and transactions are interconnected, ensuring a logical flow of data within the banking system.

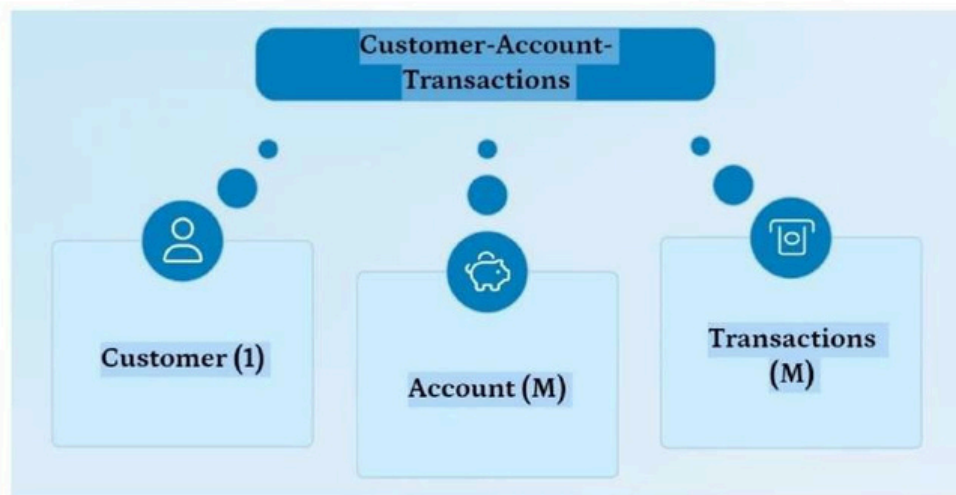


Figure 3.1: Entity-Relationship Diagram (ERD) for Banking Management

3.1.1 Table: Customer

Each customer has unique profile details and can be linked to multiple bank accounts. This ensures that a single customer can manage various financial products within the system.

Attribute	Data Type	Description
customer_id	INT (PK) AUTO_INCREMENT	Unique customer ID
name	VARCHAR(100)	Customer full name
email	VARCHAR(100)	Unique login email
phone	VARCHAR(15)	Contact number
address	VARCHAR(255)	Residential address
created_at	TIMESTAMP	Time of registration

3.1.2 Table: Account

Every account is linked to a single customer and stores crucial balance information. An account can have numerous transactions associated with it, detailing all financial activities.

Attribute	Data Type	Description
account_id	INT (PK) AUTO_INCREMENT	Unique account number
customer_id	INT (FK)	Links to customers.customer_id
account_type	VARCHAR(20)	SAVINGS / CURRENT
balance	DOUBLE	Current account balance
created_at	TIMESTAMP	Date created

3.1.3. Table: Transaction

Transactions are always linked to a specific account and record all deposit, withdrawal, or transfer activities, along with the amount and timestamp. This provides a comprehensive audit trail.

Attribute	Data Type	Description
transaction_id	INT (PK) AUTO_INCREMENT	Unique transaction ID
account_id	INT (FK)	Linked to accounts.account_id
type	VARCHAR(20)	DEPOSIT / WITHDRAW / TRANSFER
amount	DOUBLE	Transaction amount
transaction_date	TIMESTAMP	Transaction timestamp

3.2 User Authentication Module

This implicit module validates customer existence via customer ID and account existence before any transaction. It also checks for sufficient balance before processing withdrawals, ensuring only valid operations proceed. Validating whether a customer exists before creating an account.

- Validating whether an account exists before performing deposit, withdrawal, or balance checks.
- Preventing invalid inputs from being processed.
- Ensuring that operations only proceed if database records match the given customer or account ID.

3.3 Dashboard Module

The command-line interface (CLI) menu serves as the primary dashboard, offering options such as Add Customer, Create Account, Deposit, Withdraw, View Balance, and Exit. This provides easy navigation and access to core banking functions.

3.4 Customer Management Module

Acts as the foundation of the system since accounts are linked to customers. Ensures normalization and organized customer data storage. Add a new customer to the system. Store details such as first name, last name, address, phone number. Insert customer data into the customers table.

3.5 Account Management Module

Create new bank accounts for existing customers. Assign account type (Savings / Current). Store initial account balance. Ensure proper linking of customer_id to account_id using foreign keys. To enable each customer to have one or more bank accounts with secure handling.

3.6 Transaction Management Module

Deposit money into an account. Withdraw money after checking sufficient balance. Record every transaction in the transactions table. To ensure accurate and consistent financial transactions following real-world banking standards.

3.7 Balance Enquiry Module

Retrieve the current balance of an account. Display balance to the user. Validate whether the account exists. Allows customers to view their balance in real time.

3.8. Static and Base Templates Module

Although the system uses console output instead of HTML templates, it includes template-like reusable components:

- Reusable menu layout
- Standard success and error messages
- Standard JDBC connection method (DBConnection.getConnection())
- Reusable SQL execution patterns using PreparedStatement

3.9 Review / Logging Module

Automatically logs every financial transaction. Stores transaction type, amount, and timestamp.

Chapter 4

CODING

4.1 Backend Logic

```
-- Create Database
CREATE DATABASE banking_system;
USE banking_system;

-- 1. Customer Table
CREATE TABLE customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    address VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Accounts Table
CREATE TABLE accounts (
    account_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT,
    account_type VARCHAR(20) CHECK (account_type IN ('SAVINGS','CURRENT')),
    balance DOUBLE DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- 3. Transactions Table
CREATE TABLE transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    account_id INT,
    type VARCHAR(20) CHECK (type IN ('DEPOSIT','WITHDRAW','TRANSFER')),
    amount DOUBLE,
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```

        FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    );

-- 3. Transactions Table
CREATE TABLE transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    account_id INT,
    type VARCHAR(20) CHECK (type IN ('DEPOSIT','WITHDRAW','TRANSFER')),
    amount DOUBLE,
    transaction_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);

-- Sample Data
INSERT INTO customers(name,email,phone,address)
VALUES ('John Doe', 'john@gmail.com','9876543210','Delhi'),
        ('Alice Smith', 'alice@gmail.com', '9871112222', 'Mumbai');

INSERT INTO accounts(customer_id, account_type, balance)
VALUES (1,'SAVINGS',5000.00),
        (2,'CURRENT',10000.00);

```

4.2 Frontend Logic

```

package service;

import db.DBConnection;
import java.sql.*;
import model.Customer;
import model.Account;

public class BankService {

    // Add new customer
    public void addCustomer(Customer c) {
        String sql = "INSERT INTO customers (first_name, last_name, address, phone_number)
VALUES (?, ?, ?, ?)";
        try (Connection conn = DBConnection.getConnection();
            PreparedStatement stmt = conn.prepareStatement(sql)) {
            stmt.setString(1, c.getFirstName());
            stmt.setString(2, c.getLastName());
            stmt.setString(3, c.getAddress());
            stmt.setString(4, c.getPhoneNumber());
            stmt.executeUpdate();
            System.out.println("✅ Customer added successfully!");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

```

// Create account
public void createAccount(Account a) {
    String sql = "INSERT INTO accounts (customer_id, account_type, balance) VALUES (?, ?, ?)";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, a.getCustomerId());
        stmt.setString(2, a.getAccountType());
        stmt.setDouble(3, a.getBalance());
        stmt.executeUpdate();
        System.out.println("✅ Account created successfully!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// Deposit money
public void deposit(int accountId, double amount) {
    String update = "UPDATE accounts SET balance = balance + ? WHERE account_id = ?";
    String insertTransaction = "INSERT INTO transactions (account_id, transaction_type, amount)
VALUES (?, 'deposit', ?)";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement stmt1 = conn.prepareStatement(update);
        PreparedStatement stmt2 = conn.prepareStatement(insertTransaction)) {
        conn.setAutoCommit(false);

        stmt1.setDouble(1, amount);
        stmt1.setInt(2, accountId);
        stmt1.executeUpdate();

        stmt2.setInt(1, accountId);
        stmt2.setDouble(2, amount);
        stmt2.executeUpdate();

        conn.commit();
        System.out.println("✅ Deposit successful!");
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

// Withdraw money
public void withdraw(int accountId, double amount) {
    String checkBalance = "SELECT balance FROM accounts WHERE account_id = ?";
    String update = "UPDATE accounts SET balance = balance - ? WHERE account_id = ?";
    String insertTransaction = "INSERT INTO transactions (account_id, transaction_type, amount)
VALUES (?, 'withdrawal', ?)";

```

```

try (Connection conn = DBConnection.getConnection();
    PreparedStatement check = conn.prepareStatement(checkBalance);
    PreparedStatement stmt1 = conn.prepareStatement(update);
    PreparedStatement stmt2 = conn.prepareStatement(insertTransaction)) {

    check.setInt(1, accountId);
    ResultSet rs = check.executeQuery();
    if (rs.next() && rs.getDouble("balance") >= amount) {
        conn.setAutoCommit(false);

        stmt1.setDouble(1, amount);
        stmt1.setInt(2, accountId);
        stmt1.executeUpdate();

        stmt2.setInt(1, accountId);
        stmt2.setDouble(2, amount);
        stmt2.executeUpdate();

        conn.commit();
        System.out.println("✅ Withdrawal successful!");
    } else {
        System.out.println("❌ Insufficient balance!");
    }
} catch (SQLException e) {
    e.printStackTrace();
}
}

// View balance
public void viewBalance(int accountId) {
    String sql = "SELECT balance FROM accounts WHERE account_id = ?";
    try (Connection conn = DBConnection.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, accountId);
        ResultSet rs = stmt.executeQuery();
        if (rs.next()) {
            System.out.println("💰 Current Balance: $" + rs.getDouble("balance"));
        } else {
            System.out.println("❌ Account not found!");
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

Chapter 5

SCREENSHOTS

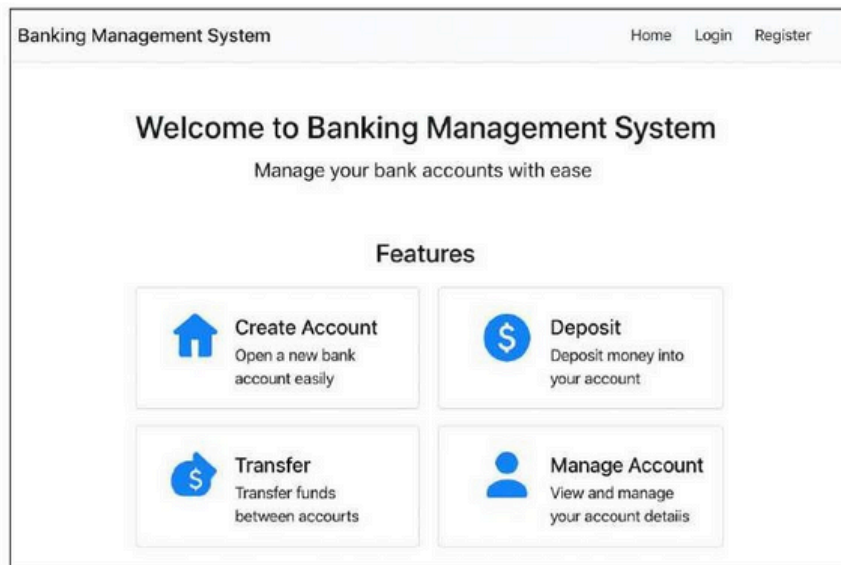


Figure 5.1: User Authentication: Login Interface

Banking Management System

HomeLogout

Create Account

Name

Initial Deposit

Email Address

Create Account

Figure 5.2: User Authentication: Create View

Banking Management System

HomeLogout

Deposit

Account Number

Amount

Deposit

Figure 5.3: User Authentication: Deposit View

Banking Management System

HomeLogout

Transfer

From Account

To Account

Amount

Transfer

Figure 5.4: User Authentication: Transfer View

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENT

The Banking Management System project effectively demonstrates the integration of Java, JDBC, and MySQL for automating core banking operations. The system efficiently supports customer creation, account management, deposits, withdrawals, and real-time balance display, ensuring secure and accurate transaction recording through a modular object-oriented approach.

6.1 Future Enhancement

- Integrate a graphical user interface (JavaFX / Swing) for improved user experience.
- Develop an Admin login panel for enhanced administrative control and system management.
- Implement an Online Fund Transfer system to facilitate seamless money transfers between accounts.
- Introduce SMS/Email notifications for transactions, providing real-time updates to customers.
- Enhance security features with password encryption and secure authentication protocols.
- Simulate ATM/UPI functionalities for broader real-world applicability.
- Provide mini-statements and detailed transaction history, with an option to convert to PDF.

REFERENCES

1. Oracle Corporation. (2023). Java Platform, Standard Edition Documentation.
2. Oracle Corporation. (2023). MySQL 8.0 Reference Manual.
3. Oracle Corporation. (2023). JDBC API Documentation.
4. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). John Wiley & Sons.
5. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
6. Horstmann, C. S., & Cornell, G. (2017). Core Java Volume I-- Fundamentals (11th ed.). Prentice Hall.
7. Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.
8. Deitel, P., & Deitel, H. (2017). Java How to Program (11th ed.). Pearson.
9. MySQL AB. (2023). MySQL Connector/J Developer Guide.
10. Eclipse Foundation. (2023). Eclipse IDE Documentation.